DAY 09: QUARKUS & PERSISTENCE

Data Bytes Technologies · Java 25 & Modern Stack

Yesterday you stood up the KYC Service skeleton, a real `POST /applicants` over a fake store. Today we give it a **database**. The same `applicants` table you queried by hand in `psql` on Day 6, now reached through Java.

// WHERE WE ARE

- **Day 6**: you talked to **PostgreSQL** directly with `psql`: `CREATE TABLE applicants`, `INSERT`, `SELECT`. You know what a row looks like.
- **Day 7**: you watched events flow through **Kafka** from the CLI.
- **Day 8**: you built the **KYC Service skeleton** in Quarkus: a validated `POST /applicants`, a `GET`, clean error mapping, a health check.

But yesterday's store was a **mock: an in-memory** `Map` inside `ApplicantService`. It forgets everything when the process dies. Today we point it at that **same** `applicants` **table** from Day 6, this time through Java.

// BY THE END OF TODAY, YOU CAN...

- name the **object-relational gap** and how an ORM closes it: Java objects ↔ SQL rows
- configure a **datasource** and let **Dev Services** hand you a real Postgres with zero config
- turn a plain class into a **JPA entity** and persist it with a **Panache repository**
- say who **owns the schema** (Flyway) and who only **validates** it (Hibernate)
- wrap a write in `@Transactional` and explain all-or-nothing in your own words
- prove a registered applicant **survives a restart**, and write a `@QuarkusTest` that talks real HTTP

// THE PROBLEM, MADE CONCRETE

`ApplicantService` holds a `Map`. Restart the process and every applicant is gone:

```
// Day 8: the "database" is a field. It dies with the JVM.
private final Map<Long, Applicant> store = new ConcurrentHashMap<>();

// Persisted data must outlive the process, that is the whole point of a database.
// We need: a place on disk (Postgres), a schema (the applicants table), and a way to
// read/write it from Java without hand-writing every INSERT and SELECT.
```

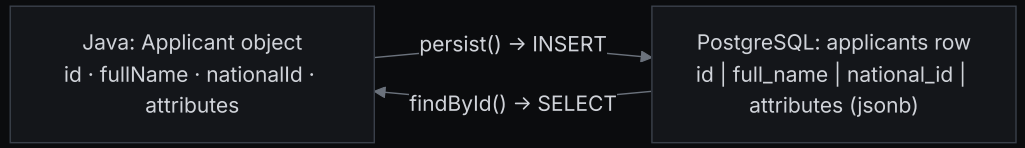
You already created that table by hand on Day 6. Today three new ideas connect it to Java: a **datasource**, an **ORM**, and **migrations** that own the schema.

// THE OBJECT-RELATIONAL GAP, IN PLAIN TERMS

> 💡 **In plain terms**: Java thinks in **objects** (an `Applicant` with fields and methods). PostgreSQL thinks in **rows** (a record in the `applicants` table). They are two different shapes for the same data, and someone has to translate between them on every read and write.

- On Day 6 *you* were the translator: you hand-wrote `INSERT INTO applicants (...) VALUES (...)`.
- That is tedious and error-prone for every field, every query, every type conversion.
- An **ORM** (Object-Relational Mapping) does the translation for you: objects in, SQL out.

// ONE PICTURE: OBJECT ↔ ROW



The arrows are the whole job of an ORM: `persist()` becomes an `INSERT`, `findById()` becomes a `SELECT`, and the column ↔ field mapping is what you declare with annotations.

// THE DATASOURCE, IN PLAIN TERMS

> 💡 **In plain terms**: a **datasource** is just "how the app finds and logs into the database": which kind of DB, at which URL, with which user and password. It is the connection, named once, that everything else borrows.

In Quarkus you tell it **one** thing for dev and test (*what kind* of database) and leave the URL blank. We will see why blank is deliberate (and magical) shortly.

// ⌨️ CODE ALONG: THE DATASOURCE CONFIG

> ⌨️ **Code along**: open `src/main/resources/application.properties` and add the persistence block with me. Four lines, no URL.

```
quarkus.datasource.db-kind=postgresql
quarkus.hibernate-orm.database.generation=none
quarkus.flyway.migrate-at-start=true
quarkus.hibernate-orm.mapping.format.global=ignore
```

Line 1 picks Postgres. Line 2: Hibernate must **not** touch the schema. Line 3: Flyway runs migrations at startup. Line 4 is today's real gotcha. We explain it when we map `jsonb`.

// WHO OWNS THE SCHEMA?

Two ways to get the `applicants` table into the database. Only one survives a team.

- **Auto-DDL**: let Hibernate guess the schema from your entities at startup. Convenient, invisible, impossible to review or reproduce.
- **Migrations**: *you* write the `CREATE TABLE` as a small SQL file, checked into Git, applied in order, recorded once. The schema becomes **versioned code**.

We choose migrations, and it is the **same DDL you already wrote in `psql` on Day 6**, now living in a file the whole team replays identically.

// FLYWAY, IN PLAIN TERMS

> 💡 **In plain terms**: Flyway is a **ledger for your schema**. Each change is a numbered file (`V1__…` , `V2__…`). On startup Flyway checks which it has already run, applies the new ones in order, and records a **checksum** so an applied file can never silently change.

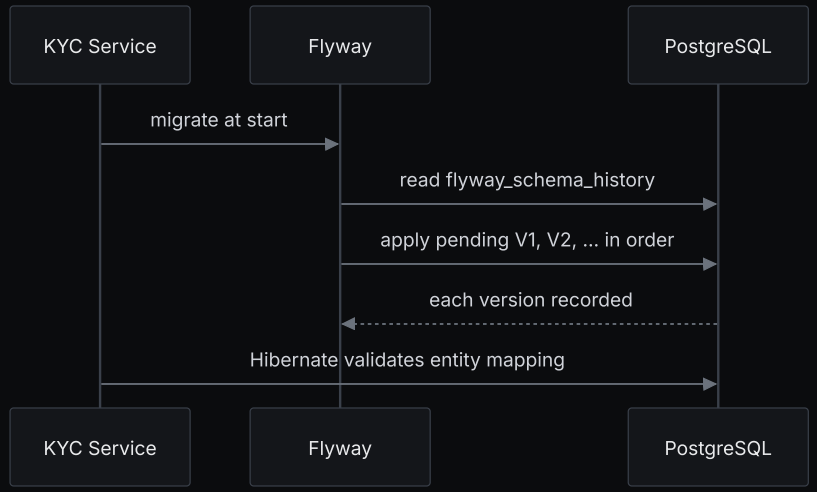
You never edit an applied migration. A change is always a *new* file. That is how the outbox table arrives **tomorrow on Day 10**.

// THE APPLICANTS TABLE: THE SAME ONE FROM DAY 6

APPLICANTS		
bigint	id	PK
varchar	full_name	
varchar	national_id	
date	date_of_birth	
varchar	country	
varchar	email	
varchar	status	
jsonb	attributes	
timestamptz	created_at	
timestamptz	updated_at	

Typed columns for what we **query and constrain**; one `jsonb` column for the genuinely open-ended rest. Choosing which is which *is* schema design.

// FLYWAY ON STARTUP



The database now has a **version history**, just like the code does, reproducible on every laptop and every environment.

// FLYWAY OWNS THE SCHEMA; HIBERNATE ONLY CHECKS

We set this once, on purpose:

```
quarkus.hibernate-orm.database.generation=none
```

> 💡 **In plain terms**: `generation=none` tells Hibernate: *do not create or alter any tables.***Flyway owns the schema.** Hibernate's only job is to **validate** that the `Applicant` entity matches the columns Flyway built.

Letting both manage DDL is a recipe for drift: two sources of truth that quietly disagree. One owner, one history.

// ⌨ CODE ALONG: THE V1 MIGRATION

⌨ **Code along**: create `src/main/resources/db/migration/V1__create_applicants.sql`. The file name *is* the version. This is your Day 6 DDL, promoted to a file.

```
create table applicants (
    id            bigint generated always as identity primary key,
    full_name     varchar(128)  not null,
    national_id   varchar(64)   not null,
    date_of_birth date          not null,
    country       varchar(64)   not null,
    email         varchar(256)  not null,
    status        varchar(16)   not null,
    attributes    jsonb         not null default '{}'::jsonb,
    created_at    timestamptz   not null default now(),
    updated_at    timestamptz   not null default now()
);
```

// NOW: HOW DOES A JAVA OBJECT BECOME A ROW?

We have the table. We have a plain `Applicant` class. The bridge is the **ORM**.

> 💡 **In plain terms**: you **annotate** a class to say "this maps to that table, this field to that column". Then saving the object becomes an `INSERT` you never hand-write.

In Quarkus the ORM is **Hibernate**, and **Panache** is a thin layer on top that makes the everyday calls short. Let’s map the entity.

// ⌨ CODE ALONG: THE `APPLICANT` ENTITY

⌨ **Code along**: annotate yesterday's plain `Applicant` so it maps to the `applicants` table.

```
@Entity
@Table(name = "applicants")
public class Applicant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long id;

    @Column(name = "full_name", nullable = false)
    public String fullName;
    // … nationalId, dateOfBirth, country, email the same …

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    public ApplicantStatus status = ApplicantStatus.PENDING;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(nullable = false, columnDefinition = "jsonb")
```

// `JSONB` MAPPING, AND TODAY'S REAL GOTCHA

The `attributes` field is a Java `Map`; the column is a JSON document. `@JdbcTypeCode(JSON)` bridges them, but Hibernate needs a **JSON mapper** to do it, and that is where the gotcha bites:

```
quarkus.hibernate-orm.mapping.format.global=ignore
```

> 💡 **In plain terms**: by default Hibernate would reuse the *same* Jackson `ObjectMapper` your REST layer uses to (de)serialise jsonb. Quarkus guards against that accidental coupling. Setting `format.global=ignore` gives Hibernate its **own** default JSON mapper, independent of REST.

// THE ENTITY FILLS IN ITS OWN TIMESTAMPS

```java
@Column(name = "created_at", nullable = false, updatable = false)
public Instant createdAt;

@PrePersist
void onCreate() {
    if (createdAt == null) createdAt = Instant.now();
    updatedAt = Instant.now();
}

@PreUpdate
void onUpdate() { updatedAt = Instant.now(); }
```

`@PrePersist` / `@PreUpdate` are **lifecycle hooks**: Hibernate calls them just before an insert or update, so the timestamps are always right without the service remembering to set them.

// TWO PANACHE STYLES: WHICH, AND WHY?

Panache gives you two ways to persist. The choice has real consequences:

- **Active Record**: persistence lives **on the entity**: `applicant.persist()`, `Applicant.findById(id)`. Wonderfully concise.
- **Repository**: the entity stays a plain mapped class; persistence moves into an injectable `ApplicantRepository`. A little more code.

The reference app chooses **Repository**, because a `static` call is impossible to fake in a test, but an injected repository is trivial to swap for a mock. You feel that payoff in the **light testing** section at the end of today.

// ⌨ CODE ALONG: THE REPOSITORY

⌨ **Code along**: one small class. `PanacheRepository<Applicant>` already gives us `persist`, `findById`, `listAll`, `delete` ... for free.

```
@ApplicationScoped
public class ApplicantRepository implements PanacheRepository<Applicant> {

    // Everything basic is inherited, we add only what's domain-specific:
    public List<Applicant> listNewest(int page, int size) {
        return findAll(Sort.by("createdAt").descending())
                .page(Page.of(page, size))
                .list();
    }
}
```

// `@TRANSACTIONAL`, IN PLAIN TERMS

> 💡 **In plain terms**: a **transaction** is all-or-nothing. `@Transactional` wraps the method so either every database change commits together, or (if anything throws) they all **roll back**, as if the method never ran.

You met transactions in raw SQL on Day 6 (`BEGIN … COMMIT`). `@Transactional` is the same idea, declared: you mark the boundary, the framework opens and commits it around your method.

// ⌨ CODE ALONG: SWAP THE STORE IN APPLICANTSERVICE

⌨ **Code along**: the heart of today. The map becomes the repository, and the method **shapes never change**, so the REST layer doesn't move a line.

```java
// AFTER, each method delegates to the repository; writes are @Transactional.
@ApplicationScoped
public class ApplicantService {

    private final ApplicantRepository applicants;

    @Inject
    public ApplicantService(ApplicantRepository applicants) {
        this.applicants = applicants;
    }

    @Transactional
    public Applicant register(Applicant draft) {
        draft.status = ApplicantStatus.PENDING;   // id + timestamps now set by DB + entity hooks
        applicants.persist(draft);
        return draft;
```

// ✋ EVERYONE WITH ME?

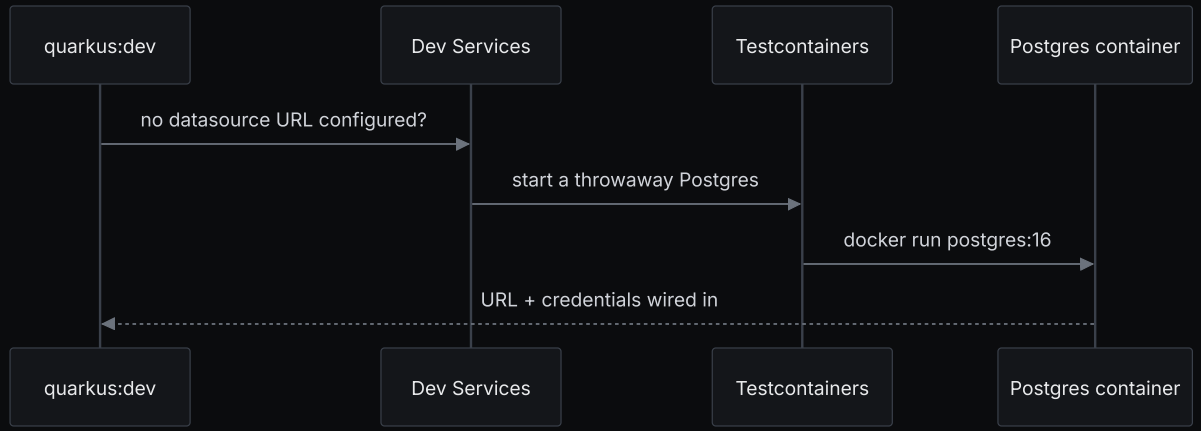
Right now you should be able to:

- say what the **object-relational gap** is and how the ORM closes it
- point at the line that makes **Flyway own the schema** and Hibernate only validate
- explain why `ApplicantService` now takes an injected `ApplicantRepository`
- say what `@Transactional` guarantees in one sentence

If any of those is fuzzy, flag it now: the run-it and testing sections build directly on these.

// BUT… WHERE'S THE DATABASE?

You wrote no JDBC URL, started no Postgres, ran no `docker run`. Yet `persist()` works.



// DEV SERVICES, IN PLAIN TERMS

> 💡 **In plain terms**: when an extension needs a service (a database, a broker) and you *haven't* configured one, **Dev Services** spins up a throwaway container for you and connects to it. Real Postgres, zero setup, gone when you stop.

It is the **Day 2 containers** idea, automated: the same Postgres image you ran by hand, now started for you on demand, and torn down after.

// 🧪 TRY IT NOW: PROVE IT PERSISTS ACROSS A RESTART (4 MIN)

With `./mvnw quarkus:dev` running, register an applicant and note its `id`:

```
curl -s localhost:8080/applicants -H 'content-type: application/json' \
  -d '{"fullName":"Ada Lovelace","nationalId":"LY-1001","dateOfBirth":"1990-05-20","country":"GB","email":"ada@ex

# Stop dev mode (Ctrl-C), start it again, then fetch that id:
curl -s localhost:8080/applicants/1 | jq
```

It's **still there**. Yesterday it would have vanished.

// LIGHT TESTING: WHY BOTHER, NOW THAT IT WORKS?

You just tested by hand: `curl`, eyeball the JSON. That works **once**. It does not re-check itself at 2am, on a teammate's laptop, or in CI.

So we add a couple of automated tests. Keep it light today: one fast **unit** test (the repository is a mockable seam) and one `@QuarkusTest` that talks real HTTP. We go deeper another day.

// `@QUARKUSTEST` BOOTS THE REAL APPLICATION

> 💡 **In plain terms:** `@QuarkusTest` starts your whole service **once** for the test class: CDI, REST, config, datasource, the lot. Your test then talks to it like a real client.

```
@QuarkusTest
class ApplicantResourceTest {
    // the app is up; REST Assured will hit its endpoints over HTTP
}
```

And here is the quiet win: that booted app gets its **own Dev Services Postgres**, automatically. The tests run against real Postgres, not a fake.

// ⌨ CODE ALONG: REST ASSURED GIVEN / WHEN / THEN

> ⌨ **Code along**: a request has three natural parts; REST Assured names them so the test reads like a spec.

```
given()
    .contentType(MediaType.APPLICATION_JSON)
    .body("""
        { "fullName": "Ada Lovelace", "nationalId": "LY-1001", "dateOfBirth": "1990-05-20", "country": "GB", "email": "
        """)
    .when().post("/applicants")
    .then().statusCode(201)
        .body("status", is("PENDING"));
```

`given` arranges the request, `when` fires it, `then` checks the response: status code **and** a field inside the JSON body. The body is a plain Java 25 **text block**, no String Templates.

// `@QuarkusTest` VS `@QuarkusIntegrationTest` (BRIEFLY)

- `@QuarkusTest`: runs against the *live application objects* in the **same JVM**. Fast, can inject beans. Use it **constantly**.
- `@QuarkusIntegrationTest`: runs the **packaged artifact** (jar, container, native) as a **black box**. Slower, but proves the thing you actually *ship* boots and serves. Use it **at the gate**.

> 💡 **In plain terms**: one tests your *code* running; the other tests your *deliverable* running. You want both, in very different amounts.

// COMMON TRAPS

- **Forgetting** `@Transactional` on a write: Panache's `persist` needs an active transaction, or it throws.
- **Leaving auto-DDL on**: if `generation` isn't `none`, Hibernate and Flyway fight over the schema and quietly drift. One owner.
- **Editing an applied migration**: Flyway's checksum rejects it at startup. A change is a **new** `V2__…` file, never an edit to `V1`.
- **Dropping the** `format.global=ignore` **line**: jsonb (de)serialisation breaks at boot. Keep Hibernate's JSON mapper separate from REST's.
- **Docker not running**: Dev Services needs it. "Cannot connect to Docker" almost always means Docker Desktop is off.

// RECAP, IN PLAIN TERMS

Today the data became real, through a few small, reviewable pieces:

- the **object-relational gap**, and the ORM that closes it, objects in, SQL out
- a **Flyway migration**: Day 6's DDL, now versioned, replayable code that owns the schema
- an **entity + repository**: Java objects mapped to rows, persistence behind one bean
- `@Transactional`: Day 6's `BEGIN…COMMIT`, declared on the method
- **Dev Services**: Day 2's containers, automated: a real Postgres you never configured

And the REST layer from yesterday? **Untouched**, because we kept the method shapes stable.

// TODAY'S LAB

🧪 **Give the KYC Service a database**: start from `git checkout checkpoint/day-08`, write the V1 migration, map the `Applicant` entity, add the `ApplicantRepository`, and wire it into `ApplicantService` behind `@Transactional`.

Brief: `labs/day-09/` · Solution to check against: `git checkout checkpoint/day-09`

Acceptance: `./mvnw -B verify` is **green**, and a `POST /applicants` **persists**, survives a restart and comes back on `GET /applicants/{id}`.

// TAKEAWAY

Persistence isn’t magic glue. It’s a **schema you version**, a **mapping you can read**, and a **transaction you declare**. Change the storage, keep the interface, and nothing above it moves.

Tomorrow, Day 10: that same `register` transaction gains one more line. It writes an event to an **outbox** so the applicant and its `applicant-registered` message commit together, then flow to **Kafka**. Persistence today is what makes that reliable tomorrow.